

Invoice & Receipt Extraction Agent — Full Implementation Plan

Target stack: Qwen2.5-VL-7B-Instruct · LangGraph · SQLite · Streamlit

GPU: NVIDIA Ada 4000 — 20GB VRAM

Goal: Feed this document to an LLM (e.g. Gemini) to generate the full codebase.

Table of Contents

1. [Project Overview](#)
 2. [Folder Structure](#)
 3. [Dependencies & Environment Setup](#)
 4. [Data Models & State Schema](#)
 5. [Module 1 — Input Handler \(PDF, Image, Screenshot\)](#)
 6. [Module 2 — Invoice Classifier \(Is this an invoice?\)](#)
 7. [Module 3 — Extraction Node \(Qwen-VL\)](#)
 8. [Module 4 — Confidence Check Router](#)
 9. [Module 5 — Re-Examine Node \(Self-Correction\)](#)
 10. [Module 6 — Human-in-the-Loop \(HITL\) Interrupt](#)
 11. [Module 7 — Storage Node \(SQLite + CSV\)](#)
 12. [LangGraph — Full Agent Assembly](#)
 13. [Streamlit UI](#)
 14. [CLI Batch Runner](#)
 15. [Database Schema](#)
 16. [Prompt Templates](#)
 17. [Configuration & Constants](#)
 18. [Error Handling Strategy](#)
 19. [Testing Plan](#)
 20. [Build Phases & Timeline](#)
-

1. Project Overview

What it does

A fully local, offline invoice extraction agent that:

- Accepts **any combination** of input: PDF files, scanned images (JPG/PNG/TIFF/WebP/BMP), screenshots, or multi-page PDFs
- **Classifies the document first** — if it is not an invoice or receipt, it immediately returns a clear rejection message without running extraction
- Extracts structured fields: vendor, date, invoice number, line items, subtotal, tax, total, currency, payment terms, and billing address
- Uses a **LangGraph agent loop** with confidence scoring per field — uncertain fields trigger a focused re-examination pass
- Escalates to a **human-in-the-loop UI** after 2 failed retries
- Saves clean JSON to **SQLite** and exports **CSV**

Non-goals

- No cloud API calls — fully offline
- No training or fine-tuning
- No multi-language OCR (can be added later)

2. Folder Structure

```

invoice_agent/
|
├─ main.py                # Entry point (CLI)
├─ agent.py               # LangGraph graph definition
├─ config.py              # All constants, thresholds, paths
├─ schema.py              # Pydantic models (state, output, DB)
|
├─ nodes/
|   ├─ __init__.py
|   ├─ input_handler.py    # PDF/image/screenshot → PIL images
|   ├─ classifier.py       # Is this an invoice? node
|   ├─ extractor.py        # Qwen-VL extraction node
|   ├─ confidence.py       # Confidence check router
|   ├─ re_examiner.py      # Focused re-examination node
|   ├─ human_interrupt.py  # HITL state preparation node
|   └─ storage.py          # SQLite + CSV export node
|
├─ models/
|   └─ __init__.py

```

```
|   └─ loader.py                # Qwen-VL model + processor singleton
|
|─ prompts/
|   └─ classify_prompt.txt      # Prompt: is this an invoice?
|   └─ extract_prompt.txt      # Prompt: full structured extraction
|   └─ reexamine_prompt.txt    # Prompt: focused single-field re-check
|
|─ ui/
|   └─ __init__.py
|   └─ streamlit_app.py        # Streamlit HITL + dashboard
|
|─ utils/
|   └─ __init__.py
|   └─ image_utils.py          # Resize, crop, enhance, convert
|   └─ json_utils.py           # Safe JSON parsing from VLM output
|
|─ db/
|   └─ invoices.db             # SQLite database (auto-created)
|
|─ exports/                    # CSV exports written here
|─ tests/
|   └─ test_classifier.py
|   └─ test_extractor.py
|   └─ sample_invoices/        # Put test files here
|
|─ requirements.txt
└─ README.md
```

3. Dependencies & Environment Setup

requirements.txt

```
torch>=2.2.0
torchvision>=0.17.0
transformers>=4.49.0
accelerate>=0.27.0
qwen-vl-utils>=0.0.8
Pillow>=10.0.0
pdf2image>=1.17.0
pymupdf>=1.23.0
pydantic>=2.5.0
```

```
langgraph>=0.1.0
langchain-core>=0.1.0
streamlit>=1.32.0
pandas>=2.1.0
sqlalchemy>=2.0.0
rich>=13.0.0
click>=8.1.0
python-magic>=0.4.27
filetype>=1.2.0
```

Environment setup instructions

```
bash

# 1. Create and activate virtual environment
python -m venv venv
source venv/bin/activate    # Linux/Mac
# or: venv\Scripts\activate # Windows

# 2. Install dependencies
pip install -r requirements.txt

# 3. Install poppler (required for pdf2image)
# Ubuntu/Debian:
sudo apt-get install poppler-utils
# Windows: download from https://github.com/oschwartz10612/poppler-windows

# 4. Download the model (first run will auto-download ~14GB)
# Model will be cached to ~/.cache/huggingface/hub/
# Model ID: Qwen/Qwen2.5-VL-7B-Instruct

# 5. Run the Streamlit UI
streamlit run ui/streamlit_app.py

# 6. Or run CLI batch mode
python main.py --input ./tests/sample_invoices/ --output ./exports/
```

4. Data Models & State Schema

`schema.py` — implement exactly as follows

```
python
```

```

from pydantic import BaseModel, Field
from typing import Optional, List, Dict, Any, Literal
from datetime import datetime
from PIL.Image import Image as PILImage

# — Line item within an invoice —————
class LineItem(BaseModel):
    description: str
    quantity: Optional[float] = None
    unit_price: Optional[float] = None
    total: Optional[float] = None
    confidence: float = Field(default=1.0, ge=0.0, le=1.0)

# — Full extracted invoice —————
class ExtractedInvoice(BaseModel):
    vendor_name: Optional[str] = None
    vendor_address: Optional[str] = None
    invoice_number: Optional[str] = None
    invoice_date: Optional[str] = None          # ISO format: YYYY-MM-DD
    due_date: Optional[str] = None
    billing_address: Optional[str] = None
    line_items: List[LineItem] = Field(default_factory=list)
    subtotal: Optional[float] = None
    tax: Optional[float] = None
    tax_rate: Optional[str] = None
    total: Optional[float] = None
    currency: str = "USD"
    payment_terms: Optional[str] = None
    notes: Optional[str] = None

    # Confidence scores per field (0.0 to 1.0)
    field_confidence: Dict[str, float] = Field(default_factory=dict)
    low_confidence_fields: List[str] = Field(default_factory=list)
    overall_confidence: float = Field(default=0.0, ge=0.0, le=1.0)

# — LangGraph agent state —————
class InvoiceState(BaseModel):
    # Input
    source_path: str          # Original file path
    source_type: Literal["pdf", "image"]  # Detected type
    images: List[Any] = Field(default_factory=list)  # PIL Images
    current_page: int = 0     # For multi-page PDFs

```

```

# Classification
is_invoice: Optional[bool] = None
classification_reason: Optional[str] = None

# Extraction
raw_llm_output: Optional[str] = None
extracted: Optional[ExtractedInvoice] = None
retry_count: int = 0
max_retries: int = 2

# Human in the loop
awaiting_human: bool = False
human_corrections: Dict[str, Any] = Field(default_factory=dict)
human_approved: bool = False

# Output
status: Literal[
    "pending",
    "not_invoice",
    "extracting",
    "low_confidence",
    "awaiting_human",
    "complete",
    "failed"
] = "pending"
error_message: Optional[str] = None
db_row_id: Optional[int] = None

class Config:
    arbitrary_types_allowed = True # Needed for PIL Image

```

— Database row model —

```

class InvoiceDBRecord(BaseModel):
    id: Optional[int] = None
    source_path: str
    source_filename: str
    processed_at: datetime
    vendor_name: Optional[str]
    invoice_number: Optional[str]
    invoice_date: Optional[str]
    due_date: Optional[str]
    total: Optional[float]
    currency: str
    overall_confidence: float

```

```
raw_json: str          # Full extracted JSON stored as string
status: str            # "complete" | "human_approved" | "low_confidence"
```

5. Module 1 — Input Handler

`nodes/input_handler.py`

Purpose: Accept any file (PDF, JPG, PNG, TIFF, WebP, BMP, screenshot) and return a list of PIL Images — one per page for PDFs, one for single images.

Implementation requirements:

```
python
```

```

# nodes/input_handler.py

import filetype
import fitz  # PyMuPDF
from pdf2image import convert_from_path
from PIL import Image, ImageEnhance
from pathlib import Path
from schema import InvoiceState
from utils.image_utils import preprocess_image

SUPPORTED_IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".tiff", ".tif", ".webp", ".b
SUPPORTED_PDF_EXTENSIONS = {".pdf"}

def detect_file_type(path: str) -> str:
    """
    Use the filetype library to detect by magic bytes (not extension).
    Returns 'pdf', 'image', or raises ValueError.
    Crucially: a screenshot saved as .png is detected as 'image'.
    """
    kind = filetype.guess(path)
    if kind is None:
        # Fallback to extension
        ext = Path(path).suffix.lower()
        if ext in SUPPORTED_PDF_EXTENSIONS:
            return "pdf"
        elif ext in SUPPORTED_IMAGE_EXTENSIONS:
            return "image"
        else:
            raise ValueError(f"Unsupported file type: {path}")

    if kind.mime == "application/pdf":
        return "pdf"
    elif kind.mime.startswith("image/"):
        return "image"
    else:
        raise ValueError(f"Unsupported MIME type: {kind.mime}")

def load_pdf_as_images(path: str, dpi: int = 200) -> list:
    """
    Convert each PDF page to a PIL Image at the given DPI.
    Use 200 DPI as default – good balance of quality and VRAM usage.
    Falls back to PyMuPDF if pdf2image fails (e.g. encrypted PDFs).

```

```

"""
try:
    images = convert_from_path(path, dpi=dpi, fmt="RGB")
    return [preprocess_image(img) for img in images]
except Exception:
    # Fallback: PyMuPDF
    doc = fitz.open(path)
    images = []
    for page in doc:
        mat = fitz.Matrix(dpi / 72, dpi / 72)
        pix = page.get_pixmap(matrix=mat, colorspace=fitz.csRGB)
        img = Image.frombytes("RGB", [pix.width, pix.height], pix.samples)
        images.append(preprocess_image(img))
    return images

def load_image(path: str) -> list:
    """
    Load a single image file. Returns a list with one PIL Image.
    Handles EXIF rotation, converts to RGB (strips alpha).
    """
    img = Image.open(path)
    # Handle EXIF orientation
    try:
        from PIL import ImageOps
        img = ImageOps.exif_transpose(img)
    except Exception:
        pass
    # Convert to RGB (handles RGBA PNGs, CMYK scans, etc.)
    if img.mode != "RGB":
        img = img.convert("RGB")
    return [preprocess_image(img)]

def input_handler_node(state: InvoiceState) -> InvoiceState:
    """
    LangGraph node: loads the source file into a list of PIL Images.
    Updates state.images and state.source_type.
    """
    try:
        source_type = detect_file_type(state.source_path)
        state.source_type = source_type

        if source_type == "pdf":

```

```
        state.images = load_pdf_as_images(state.source_path)
    else:
        state.images = load_image(state.source_path)

    if not state.images:
        raise ValueError("No images could be loaded from the file.")

except Exception as e:
    state.status = "failed"
    state.error_message = f"Input handler error: {str(e)}"

return state
```

utils/image_utils.py

python

```

# utils/image_utils.py
from PIL import Image, ImageEnhance, ImageFilter

MAX_LONG_SIDE = 1280 # Qwen-VL optimal resolution

def preprocess_image(img: Image.Image) -> Image.Image:
    """
    Resize image so the longest side is MAX_LONG_SIDE.
    Lightly enhance contrast for scanned documents.
    Returns a clean RGB PIL Image.
    """
    # Resize keeping aspect ratio
    w, h = img.size
    long_side = max(w, h)
    if long_side > MAX_LONG_SIDE:
        scale = MAX_LONG_SIDE / long_side
        new_w = int(w * scale)
        new_h = int(h * scale)
        img = img.resize((new_w, new_h), Image.LANCZOS)

    # Light contrast enhancement for scans
    enhancer = ImageEnhance.Contrast(img)
    img = enhancer.enhance(1.2)

    return img

def crop_region(img: Image.Image, region: str) -> Image.Image:
    """
    Crop a named region of the image for focused re-examination.

    Regions:
        "top"      - top 35% (vendor name, header, invoice number, date)
        "middle"   - middle 40% (line items)
        "bottom"   - bottom 35% (totals, tax, payment terms)
        "full"     - entire image (fallback)
    """
    w, h = img.size
    if region == "top":
        return img.crop((0, 0, w, int(h * 0.35)))
    elif region == "middle":
        return img.crop((0, int(h * 0.30), w, int(h * 0.70)))
    elif region == "bottom":

```

```
        return img.crop((0, int(h * 0.65), w, h))
    else:
        return img
```

```
# Maps each extractable field to its likely image region
```

```
FIELD_TO_REGION = {
    "vendor_name": "top",
    "vendor_address": "top",
    "invoice_number": "top",
    "invoice_date": "top",
    "due_date": "top",
    "billing_address": "top",
    "line_items": "middle",
    "subtotal": "bottom",
    "tax": "bottom",
    "tax_rate": "bottom",
    "total": "bottom",
    "currency": "bottom",
    "payment_terms": "bottom",
}
```

6. Module 2 — Invoice Classifier

`nodes/classifier.py`

Purpose: Before any extraction, determine whether the loaded image is actually an invoice or receipt. This is the most important gate — it prevents wasted compute and gives users a clear rejection message.

Implementation requirements:

```
python
```

```
# nodes/classifier.py
```

```
from models.loader import get_model_and_processor
from schema import InvoiceState
from utils.json_utils import safe_parse_json
```

```
CLASSIFY_PROMPT = """
```

```
You are a document classification expert.
```

```
Examine this document image carefully and determine if it is an invoice, bill, or re
```

```
An invoice or receipt MUST contain at least TWO of the following:
```

1. A vendor/company name or logo
2. A monetary amount or price
3. A date
4. An invoice number, order number, or receipt number
5. Line items, services, or products listed with prices
6. Billing or payment information
7. Words like "Invoice", "Receipt", "Bill", "Statement", "Order"

```
Respond ONLY with a JSON object in this exact format (no other text):
```

```
{
  "is_invoice": true or false,
  "confidence": 0.0 to 1.0,
  "document_type": "invoice" | "receipt" | "bill" | "bank_statement" | "photo" | "sc
  "reason": "brief explanation in one sentence"
}
```

```
Examples of what is NOT an invoice:
```

- A photo of a person, object, or scene
- A screenshot of a website or app UI
- A blank page or test page
- A letter or memo without billing info
- A product catalog without per-item pricing to a specific buyer

```
"""
```

```
def classifier_node(state: InvoiceState) -> InvoiceState:
```

```
    """
```

```
    LangGraph node: classifies whether the document is an invoice.
    Uses the first page only (most informative for classification).
    Updates state.is_invoice and state.classification_reason.
```

```
    """
```

```
    if state.status == "failed" or not state.images:
```

```
    return state
```

```
model, processor = get_model_and_processor()
```

```
image = state.images[0] # Use first page for classification
```

```
try:
```

```
    messages = [
```

```
        {
```

```
            "role": "user",
```

```
            "content": [
```

```
                {"type": "image", "image": image},
```

```
                {"type": "text", "text": CLASSIFY_PROMPT},
```

```
            ],
```

```
        }
```

```
    ]
```

```
    text = processor.apply_chat_template(
```

```
        messages, tokenize=False, add_generation_prompt=True
```

```
    )
```

```
    image_inputs, video_inputs = process_vision_info(messages)
```

```
    inputs = processor(
```

```
        text=[text],
```

```
        images=image_inputs,
```

```
        videos=video_inputs,
```

```
        padding=True,
```

```
        return_tensors="pt",
```

```
    ).to(model.device)
```

```
    with torch.no_grad():
```

```
        generated_ids = model.generate(
```

```
            **inputs,
```

```
            max_new_tokens=200,
```

```
            temperature=0.1, # Low temperature for consistent classification
```

```
            do_sample=True,
```

```
        )
```

```
    generated_ids_trimmed = [
```

```
        out_ids[len(in_ids):]
```

```
        for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
```

```
    ]
```

```
    output_text = processor.batch_decode(
```

```
        generated_ids_trimmed,
```

```
        skip_special_tokens=True,
```

```
        clean_up_tokenization_spaces=False,
```

```

)[0]

result = safe_parse_json(output_text)

if result:
    state.is_invoice = result.get("is_invoice", False)
    state.classification_reason = result.get("reason", "Unknown")
    doc_type = result.get("document_type", "unknown")

    if not state.is_invoice:
        state.status = "not_invoice"
        state.error_message = (
            f"This document does not appear to be an invoice or receipt. "
            f"Detected type: '{doc_type}'. Reason: {state.classification_reason}"
        )
    else:
        # If JSON parsing fails, default to attempting extraction
        # (fail open rather than fail closed)
        state.is_invoice = True
        state.classification_reason = "Classification response could not be parsed"

except Exception as e:
    # On any model error, fail open
    state.is_invoice = True
    state.classification_reason = f"Classification error (proceeding): {str(e)}"

return state

```

7. Module 3 — Extraction Node

`nodes/extractor.py`

Purpose: Run the full structured extraction using Qwen-VL. Output must be a valid JSON object with confidence scores per field.

python

```
# nodes/extractor.py
```

```
import torch
from qwen_vl_utils import process_vision_info
from models.loader import get_model_and_processor
from schema import InvoiceState, ExtractedInvoice, LineItem
from utils.json_utils import safe_parse_json
from config import CONFIDENCE_THRESHOLD
```

```
EXTRACT_PROMPT = """
```

```
You are an expert invoice data extraction system.
```

```
Extract ALL information from this invoice image into a structured JSON object.
```

```
For each field, you MUST also provide a confidence score between 0.0 and 1.0:
```

- 1.0 = you can clearly read this value with complete certainty
- 0.7-0.9 = you can read it but some characters may be slightly unclear
- 0.5-0.7 = the value is partially readable or inferred
- below 0.5 = you cannot confidently determine this value

```
Respond ONLY with a valid JSON object in this exact format (no markdown, no extra te
```

```
{
  "vendor_name": "string or null",
  "vendor_address": "string or null",
  "invoice_number": "string or null",
  "invoice_date": "YYYY-MM-DD or null",
  "due_date": "YYYY-MM-DD or null",
  "billing_address": "string or null",
  "line_items": [
    {
      "description": "string",
      "quantity": number or null,
      "unit_price": number or null,
      "total": number or null,
      "confidence": 0.0-1.0
    }
  ],
  "subtotal": number or null,
  "tax": number or null,
  "tax_rate": "string or null (e.g. '10%' or '20%')",
  "total": number or null,
  "currency": "3-letter code e.g. USD, GBP, EUR, PKR",
```

```

"payment_terms": "string or null",
"notes": "string or null",
"field_confidence": {
    "vendor_name": 0.0-1.0,
    "vendor_address": 0.0-1.0,
    "invoice_number": 0.0-1.0,
    "invoice_date": 0.0-1.0,
    "due_date": 0.0-1.0,
    "billing_address": 0.0-1.0,
    "subtotal": 0.0-1.0,
    "tax": 0.0-1.0,
    "total": 0.0-1.0,
    "currency": 0.0-1.0,
    "payment_terms": 0.0-1.0
}
}

```

IMPORTANT:

- Dates MUST be in YYYY-MM-DD format. If you see "15 Apr 2025" convert it to "2025-04-15".
- Monetary values MUST be plain numbers (no currency symbols). E.g. 1240.50 not "\$1,240.50".
- If a field is not present in the invoice, use null.
- For currency, detect from symbols: \$ = USD, £ = GBP, € = EUR, ₨ or Rs = PKR, ¥ = JPY, etc.

```

def extractor_node(state: InvoiceState) -> InvoiceState:
    """
    LangGraph node: runs Qwen-VL extraction on the invoice image(s).
    For multi-page PDFs, processes the first page by default.
    (Multi-page handling can be extended later.)
    """
    if state.status in ("failed", "not_invoice") or not state.images:
        return state

    model, processor = get_model_and_processor()
    state.status = "extracting"

    try:
        # For multi-page PDFs, use page index (default 0)
        image = state.images[state.current_page]

        messages = [
            {
                "role": "user",

```

```

        "content": [
            {"type": "image", "image": image},
            {"type": "text", "text": EXTRACT_PROMPT},
        ],
    }
]

text = processor.apply_chat_template(
    messages, tokenize=False, add_generation_prompt=True
)
image_inputs, video_inputs = process_vision_info(messages)
inputs = processor(
    text=[text],
    images=image_inputs,
    videos=video_inputs,
    padding=True,
    return_tensors="pt",
).to(model.device)

with torch.no_grad():
    generated_ids = model.generate(
        **inputs,
        max_new_tokens=1500,
        temperature=0.1,
        do_sample=True,
    )

generated_ids_trimmed = [
    out_ids[len(in_ids):]
    for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
]
output_text = processor.batch_decode(
    generated_ids_trimmed,
    skip_special_tokens=True,
    clean_up_tokenization_spaces=False,
)[0]

state.raw_llm_output = output_text
parsed = safe_parse_json(output_text)

if parsed:
    state.extracted = _build_extracted_invoice(parsed)
else:
    state.retry_count += 1

```

```
state.status = "low_confidence"
state.error_message = "Could not parse extraction output as JSON."
```

```
except Exception as e:
    state.status = "failed"
    state.error_message = f"Extraction error: {str(e)}"
```

```
return state
```

```
def _build_extracted_invoice(parsed: dict) -> ExtractedInvoice:
    """Build an ExtractedInvoice from the raw parsed dict and compute overall confidence"""
    line_items = []
    for item in parsed.get("line_items", []):
        line_items.append(LineItem(
            description=item.get("description", ""),
            quantity=item.get("quantity"),
            unit_price=item.get("unit_price"),
            total=item.get("total"),
            confidence=item.get("confidence", 1.0),
        ))

    field_confidence = parsed.get("field_confidence", {})
    low_conf_fields = [
        field for field, score in field_confidence.items()
        if score < CONFIDENCE_THRESHOLD
    ]

    confidences = list(field_confidence.values())
    overall = sum(confidences) / len(confidences) if confidences else 0.0

    return ExtractedInvoice(
        vendor_name=parsed.get("vendor_name"),
        vendor_address=parsed.get("vendor_address"),
        invoice_number=parsed.get("invoice_number"),
        invoice_date=parsed.get("invoice_date"),
        due_date=parsed.get("due_date"),
        billing_address=parsed.get("billing_address"),
        line_items=line_items,
        subtotal=parsed.get("subtotal"),
        tax=parsed.get("tax"),
        tax_rate=parsed.get("tax_rate"),
        total=parsed.get("total"),
        currency=parsed.get("currency", "USD"),
```

```
        payment_terms=parsed.get("payment_terms"),
        notes=parsed.get("notes"),
        field_confidence=field_confidence,
        low_confidence_fields=low_conf_fields,
        overall_confidence=overall,
    )
```

8. Module 4 — Confidence Check Router

`nodes/confidence.py`

```
python
```

```

# nodes/confidence.py

from schema import InvoiceState
from config import CONFIDENCE_THRESHOLD, MAX_RETRIES

def confidence_router(state: InvoiceState) -> str:
    """
    LangGraph conditional edge function.
    Returns the name of the next node to route to.

    Routing logic:
    - Not an invoice                -> "end_not_invoice"
    - Failed state                  -> "end_failed"
    - No extracted data             -> "re_examine" (retry)
    - Low confidence fields
      AND retries remaining         -> "re_examine"
      AND retries exhausted         -> "human_interrupt"
    - All fields high conf.         -> "structure_output"
    - Human already approved        -> "storage"
    """
    if state.status == "not_invoice":
        return "end_not_invoice"

    if state.status == "failed":
        return "end_failed"

    if state.human_approved:
        return "storage"

    if state.extracted is None:
        if state.retry_count < MAX_RETRIES:
            return "re_examine"
        else:
            return "human_interrupt"

    has_low_confidence = len(state.extracted.low_confidence_fields) > 0

    if not has_low_confidence:
        return "storage"

    if state.retry_count < MAX_RETRIES:
        return "re_examine"

```

```
else:  
    return "human_interrupt"
```

9. Module 5 — Re-Examine Node

`nodes/re_examiner.py`

Purpose: For each low-confidence field, crop the relevant image region and send a laser-focused, single-field prompt to the model. This dramatically improves accuracy on difficult fields.

python

```
# nodes/re_examiner.py
```

```
import torch
from qwen_vl_utils import process_vision_info
from models.loader import get_model_and_processor
from schema import InvoiceState
from utils.image_utils import crop_region, FIELD_TO_REGION
from utils.json_utils import safe_parse_json
```

```
REEXAMINE_PROMPT_TEMPLATE = """
Focus only on extracting the {field_name} from this cropped section of an invoice.
```

```
{field_instructions}
```

```
Respond ONLY with a JSON object:
```

```
{{
  "value": <the extracted value or null>,
  "confidence": <0.0 to 1.0>
}}
```

```
FIELD_INSTRUCTIONS = {
    "vendor_name": "What is the company or vendor name? Look for the largest text, 1
    "invoice_number": "What is the invoice number? Look for 'Invoice #', 'INV-', 'In
    "invoice_date": "What is the invoice date? Return in YYYY-MM-DD format. Look for
    "due_date": "What is the payment due date? Return in YYYY-MM-DD format. Look for
    "total": "What is the final total amount? Return as a plain number only (e.g. 12
    "subtotal": "What is the subtotal before tax? Return as a plain number only.",
    "tax": "What is the tax amount? Return as a plain number only.",
    "tax_rate": "What is the tax rate percentage? Return as a string like '10%' or '
    "currency": "What currency is this invoice in? Return the 3-letter code: USD, GB
    "payment_terms": "What are the payment terms? E.g. 'Net 30', 'Due on receipt', e
}
```

```
def re_examiner_node(state: InvoiceState) -> InvoiceState:
```

```
    """
```

```
    LangGraph node: re-examines each low-confidence field individually.
```

```
    Crops the image to the relevant region and uses a focused prompt.
```

```
    Updates the extracted fields and confidence scores in state.
```

```
    """
```

```
    if state.status in ("failed", "not_invoice") or not state.images:
```

```
        return state
```

```

if state.extracted is None:
    state.retry_count += 1
    return state

model, processor = get_model_and_processor()
image = state.images[state.current_page]
low_conf_fields = state.extracted.low_confidence_fields.copy()

for field in low_conf_fields:
    if field not in FIELD_INSTRUCTIONS:
        continue

    region_name = FIELD_TO_REGION.get(field, "full")
    cropped_image = crop_region(image, region_name)
    instructions = FIELD_INSTRUCTIONS[field]
    prompt = REEXAMINE_PROMPT_TEMPLATE.format(
        field_name=field.replace("_", " ").title(),
        field_instructions=instructions,
    )

    messages = [
        {
            "role": "user",
            "content": [
                {"type": "image", "image": cropped_image},
                {"type": "text", "text": prompt},
            ],
        }
    ]

    try:
        text = processor.apply_chat_template(
            messages, tokenize=False, add_generation_prompt=True
        )
        image_inputs, video_inputs = process_vision_info(messages)
        inputs = processor(
            text=[text],
            images=image_inputs,
            videos=video_inputs,
            padding=True,
            return_tensors="pt",
        ).to(model.device)

```

```

with torch.no_grad():
    generated_ids = model.generate(
        **inputs,
        max_new_tokens=100,
        temperature=0.1,
        do_sample=True,
    )

    generated_ids_trimmed = [
        out_ids[len(in_ids):]
        for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
    ]
    output_text = processor.batch_decode(
        generated_ids_trimmed,
        skip_special_tokens=True,
        clean_up_tokenization_spaces=False,
    )[0]

    result = safe_parse_json(output_text)
    if result and "value" in result:
        new_value = result["value"]
        new_confidence = result.get("confidence", 0.5)

        # Update the extracted field value
        if hasattr(state.extracted, field) and new_value is not None:
            setattr(state.extracted, field, new_value)
            state.extracted.field_confidence[field] = new_confidence

    except Exception:
        # If re-examination fails for a field, leave the original value
        continue

# Recompute low-confidence fields and overall confidence
from config import CONFIDENCE_THRESHOLD
state.extracted.low_confidence_fields = [
    f for f, score in state.extracted.field_confidence.items()
    if score < CONFIDENCE_THRESHOLD
]
scores = list(state.extracted.field_confidence.values())
state.extracted.overall_confidence = sum(scores) / len(scores) if scores else 0.
state.retry_count += 1

return state

```

10. Module 6 — Human-in-the-Loop Interrupt

nodes/human_interrupt.py

```
python
```

```

# nodes/human_interrupt.py

from schema import InvoiceState

def human_interrupt_node(state: InvoiceState) -> InvoiceState:
    """
    LangGraph node: flags the state as awaiting human review.
    The actual UI interaction happens in Streamlit (streamlit_app.py).
    This node sets the flag; LangGraph's interrupt_before mechanism
    pauses graph execution here.
    """
    state.awaiting_human = True
    state.status = "awaiting_human"
    return state

def apply_human_corrections(state: InvoiceState, corrections: dict) -> InvoiceState:
    """
    Called by the Streamlit UI after the human submits corrections.
    Applies corrections to extracted fields and marks as approved.

    corrections: dict mapping field_name → corrected_value
    e.g. {"total": 1240.50, "invoice_date": "2025-04-15"}
    """
    if state.extracted is None:
        from schema import ExtractedInvoice
        state.extracted = ExtractedInvoice()

    for field, value in corrections.items():
        if hasattr(state.extracted, field):
            setattr(state.extracted, field, value)
            # Human-corrected fields get maximum confidence
            state.extracted.field_confidence[field] = 1.0

    # Recompute low-confidence fields after corrections
    from config import CONFIDENCE_THRESHOLD
    state.extracted.low_confidence_fields = [
        f for f, score in state.extracted.field_confidence.items()
        if score < CONFIDENCE_THRESHOLD
    ]

    state.awaiting_human = False
    state.human_approved = True

```

```
state.status = "complete"  
return state
```

11. Module 7 — Storage Node

nodes/storage.py

python

```

# nodes/storage.py

import json
import sqlite3
import pandas as pd
from pathlib import Path
from datetime import datetime
from schema import InvoiceState, InvoiceDBRecord
from config import DB_PATH, EXPORTS_DIR

def init_db():
    """Create database tables if they don't exist."""
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("""
        CREATE TABLE IF NOT EXISTS invoices (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            source_path TEXT NOT NULL,
            source_filename TEXT NOT NULL,
            processed_at TEXT NOT NULL,
            vendor_name TEXT,
            invoice_number TEXT,
            invoice_date TEXT,
            due_date TEXT,
            total REAL,
            currency TEXT DEFAULT 'USD',
            overall_confidence REAL,
            raw_json TEXT NOT NULL,
            status TEXT NOT NULL
        )
    """)
    c.execute("""
        CREATE TABLE IF NOT EXISTS line_items (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            invoice_id INTEGER NOT NULL,
            description TEXT,
            quantity REAL,
            unit_price REAL,
            total REAL,
            confidence REAL,
            FOREIGN KEY (invoice_id) REFERENCES invoices(id)
        )
    """)

```

```
conn.commit()
conn.close()
```

```
def storage_node(state: InvoiceState) -> InvoiceState:
    """
    LangGraph node: saves extracted invoice to SQLite.
    Also updates state.status to 'complete'.
    """
    if state.extracted is None or state.status in ("failed", "not_invoice"):
        return state

    init_db()
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()

    extracted = state.extracted
    status_label = "human_approved" if state.human_approved else "complete"

    # Insert invoice header
    c.execute("""
        INSERT INTO invoices
        (source_path, source_filename, processed_at, vendor_name, invoice_number,
         invoice_date, due_date, total, currency, overall_confidence, raw_json, stat
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        state.source_path,
        Path(state.source_path).name,
        datetime.now().isoformat(),
        extracted.vendor_name,
        extracted.invoice_number,
        extracted.invoice_date,
        extracted.due_date,
        extracted.total,
        extracted.currency,
        extracted.overall_confidence,
        extracted.model_dump_json(),
        status_label,
    ))

    invoice_id = c.lastrowid

    # Insert line items
    for item in extracted.line_items:
```

```

        c.execute("""
            INSERT INTO line_items
            (invoice_id, description, quantity, unit_price, total, confidence)
            VALUES (?, ?, ?, ?, ?, ?)
        """, (
            invoice_id,
            item.description,
            item.quantity,
            item.unit_price,
            item.total,
            item.confidence,
        ))

    conn.commit()
    conn.close()

    state.db_row_id = invoice_id
    state.status = "complete"
    return state

def export_to_csv(output_path: str = None) -> str:
    """
    Export all invoices from SQLite to CSV.
    Returns the path of the exported file.
    """
    init_db()
    conn = sqlite3.connect(DB_PATH)
    df = pd.read_sql_query("SELECT * FROM invoices", conn)
    conn.close()

    if output_path is None:
        Path(EXPORTS_DIR).mkdir(parents=True, exist_ok=True)
        output_path = str(
            Path(EXPORTS_DIR) / f"invoices_export_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
        )

    df.to_csv(output_path, index=False)
    return output_path

```

python

```

# agent.py

from langgraph.graph import StateGraph, END
from langgraph.checkpoint.sqlite import SqliteSaver
from schema import InvoiceState
from nodes.input_handler import input_handler_node
from nodes.classifier import classifier_node
from nodes.extractor import extractor_node
from nodes.confidence import confidence_router
from nodes.re_examiner import re_examiner_node
from nodes.human_interrupt import human_interrupt_node
from nodes.storage import storage_node
from config import DB_PATH

def build_graph():
    """
    Build and compile the LangGraph invoice extraction agent.

    Graph structure:

    input_handler
      ↓
    classifier —[not_invoice]→ END (with rejection message)
      ↓ [is_invoice]
    extractor
      ↓
    confidence_router —[high confidence]→ storage → END
      ↓ [low confidence + retries left]
    re_examiner
      ↓
    confidence_router (loop back)
      ↓ [retries exhausted]
    human_interrupt (LangGraph pauses here for HITL)
      ↓ [after human approval]
    storage → END
    """

    # Use SQLite checkpointer so state persists across HITL interrupts
    checkpointer = SqliteSaver.from_conn_string(DB_PATH)

    builder = StateGraph(InvoiceState)

    # Add nodes

```

```

builder.add_node("input_handler", input_handler_node)
builder.add_node("classifier", classifier_node)
builder.add_node("extractor", extractor_node)
builder.add_node("re_examiner", re_examiner_node)
builder.add_node("human_interrupt", human_interrupt_node)
builder.add_node("storage", storage_node)

# Set entry point
builder.set_entry_point("input_handler")

# Edges: linear flow
builder.add_edge("input_handler", "classifier")
builder.add_edge("extractor", "confidence_check") # named conditional below

# Conditional edge after classifier
builder.add_conditional_edges(
    "classifier",
    lambda state: "extractor" if state.is_invoice else END,
    {
        "extractor": "extractor",
        END: END,
    }
)

# Conditional edge after extractor + after re_examiner (same router)
for source in ["extractor", "re_examiner"]:
    builder.add_conditional_edges(
        source,
        confidence_router,
        {
            "re_examine": "re_examiner",
            "human_interrupt": "human_interrupt",
            "storage": "storage",
            "end_not_invoice": END,
            "end_failed": END,
        }
    )

# After human approval, route to storage
builder.add_conditional_edges(
    "human_interrupt",
    confidence_router,
    {
        "storage": "storage",

```

```

        "end_failed": END,
    }
)

builder.add_edge("storage", END)

# interrupt_before: pause execution at human_interrupt node
graph = builder.compile(
    checkpointer=checkpointer,
    interrupt_before=["human_interrupt"],
)

return graph

def run_invoice(file_path: str, thread_id: str = None) -> InvoiceState:
    """
    Run the invoice agent on a single file.
    Returns the final InvoiceState.

    thread_id is used for checkpointing – pass the same thread_id
    to resume after a human-in-the-loop interrupt.
    """
    import uuid
    if thread_id is None:
        thread_id = str(uuid.uuid4())

    graph = build_graph()
    config = {"configurable": {"thread_id": thread_id}}

    initial_state = InvoiceState(source_path=file_path)
    final_state = None

    for event in graph.stream(initial_state, config=config):
        final_state = event

    return final_state, thread_id

```

13. Streamlit UI

`ui/streamlit_app.py`

Full requirements for the Streamlit app:

```
python
```

```

# ui/streamlit_app.py
# Run with: streamlit run ui/streamlit_app.py

import streamlit as st
import sys
import os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import json
import pandas as pd
from pathlib import Path
from PIL import Image

from agent import run_invoice, build_graph
from nodes.human_interrupt import apply_human_corrections
from nodes.storage import export_to_csv, init_db
from config import DB_PATH

st.set_page_config(
    page_title="Invoice Extraction Agent",
    page_icon="📄",
    layout="wide"
)

# — Sidebar navigation —
page = st.sidebar.radio("Navigate", ["Extract Invoice", "Review Queue", "Database",

# — PAGE 1: EXTRACT —
if page == "Extract Invoice":
    st.title("📄 Invoice Extraction Agent")
    st.caption("Upload a PDF, JPG, PNG, screenshot, or scanned image")

    uploaded_file = st.file_uploader(
        "Drop your file here",
        type=["pdf", "jpg", "jpeg", "png", "tiff", "tif", "webp", "bmp"],
        help="Supports PDFs, scanned invoices, photos, and screenshots"
    )

    if uploaded_file:
        # Save to temp file
        import tempfile
        suffix = Path(uploaded_file.name).suffix
        with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as tmp:

```

```

tmp.write(uploaded_file.read())
tmp_path = tmp.name

col1, col2 = st.columns([1, 1])

with col1:
    st.subheader("Input Document")
    if suffix.lower() == ".pdf":
        st.info("PDF uploaded – showing first page preview")
        from pdf2image import convert_from_path
        pages = convert_from_path(tmp_path, dpi=100)
        if pages:
            st.image(pages[0], use_container_width=True)
    else:
        st.image(tmp_path, use_container_width=True)

with col2:
    st.subheader("Extraction Result")

    if st.button("▶ Run Extraction", type="primary", use_container_width=True):
        with st.spinner("Classifying document..."):
            final_state, thread_id = run_invoice(tmp_path)
            st.session_state["last_state"] = final_state
            st.session_state["last_thread_id"] = thread_id

    if "last_state" in st.session_state:
        state = st.session_state["last_state"]

        # — NOT AN INVOICE —
        if state.status == "not_invoice":
            st.error("⚠ This document does not appear to be an invoice.")
            st.info(f"***Reason:** {state.error_message}")
            st.stop()

        # — FAILED —
        elif state.status == "failed":
            st.error(f"❌ Extraction failed: {state.error_message}")

        # — AWAITING HUMAN —
        elif state.status == "awaiting_human":
            st.warning("⚠ Low confidence detected. Please review and correct")
            _render_hitl_form(state, st.session_state["last_thread_id"])

        # — COMPLETE —

```

```

        elif state.status == "complete":
            _render_results(state)

def _render_results(state):
    """Render extracted invoice fields with confidence indicators."""
    ext = state.extracted
    if ext is None:
        st.warning("No data extracted.")
        return

    conf_color = "green" if ext.overall_confidence >= 0.85 else "orange" if ext.over
    st.markdown(f"""**Overall confidence:** {conf_color}[{ext.overall_confidence:.0%}]

    st.markdown("----")

    # Header fields
    cols = st.columns(2)
    with cols[0]:
        _field("Vendor", ext.vendor_name, ext.field_confidence.get("vendor_name"))
        _field("Invoice #", ext.invoice_number, ext.field_confidence.get("invoice_nu
        _field("Date", ext.invoice_date, ext.field_confidence.get("invoice_date"))
        _field("Due Date", ext.due_date, ext.field_confidence.get("due_date"))
    with cols[1]:
        _field("Total", f"{ext.currency} {ext.total}", ext.field_confidence.get("tot
        _field("Subtotal", f"{ext.currency} {ext.subtotal}", ext.field_confidence.ge
        _field("Tax", f"{ext.currency} {ext.tax} ({ext.tax_rate or ''})", ext.field_
        _field("Payment Terms", ext.payment_terms, ext.field_confidence.get("payment

    if ext.line_items:
        st.markdown("""**Line Items**""")
        df = pd.DataFrame([item.model_dump() for item in ext.line_items])
        st.dataframe(df, use_container_width=True)

    with st.expander("Raw JSON"):
        st.json(json.loads(ext.model_dump_json()))

    if state.db_row_id:
        st.success(f"✅ Saved to database — row ID: {state.db_row_id}")

def _field(label: str, value, confidence: float = None):
    """Render a single field with a confidence badge."""
    if value is None or str(value).strip() in ("None", ""):

```

```

        badge = "🟡"
        value_str = "-"
    elif confidence is not None and confidence < 0.75:
        badge = "🟡"
        value_str = str(value)
    else:
        badge = "🟢"
        value_str = str(value)
    st.markdown(f"**{label}** {badge} \n{value_str}")

```

```

def _render_hitl_form(state, thread_id):
    """Render editable form for human-in-the-loop corrections."""
    ext = state.extracted

    st.markdown("**Fields requiring review (yellow = low confidence):**")

    corrections = {}
    with st.form("hitl_form"):
        corrections["vendor_name"] = st.text_input("Vendor Name", value=ext.vendor_name)
        corrections["invoice_number"] = st.text_input("Invoice Number", value=ext.invoice_number)
        corrections["invoice_date"] = st.text_input("Invoice Date (YYYY-MM-DD)", value=ext.invoice_date)
        corrections["total"] = st.number_input("Total Amount", value=float(ext.total_amount))
        corrections["currency"] = st.text_input("Currency", value=ext.currency or "USD")

    submitted = st.form_submit_button("✅ Approve & Save", type="primary")
    if submitted:
        updated_state = apply_human_corrections(state, corrections)
        from nodes.storage import storage_node
        final = storage_node(updated_state)
        st.session_state["last_state"] = final
        st.success(f"✅ Saved with human corrections — row ID: {final.db_row_id}")
        st.rerun()

```

```

# — PAGE 2: REVIEW QUEUE —————
elif page == "Review Queue":
    st.title("🔍 Review Queue")
    st.caption("Items awaiting human approval")
    st.info("Human-in-the-loop items will appear here after automated retries are exhausted")

```

```

# — PAGE 3: DATABASE —————
elif page == "Database":

```

```
st.title("📄 Invoice Database")
init_db()
import sqlite3
conn = sqlite3.connect(DB_PATH)
df = pd.read_sql_query("SELECT id, source_filename, vendor_name, invoice_number,
conn.close()
if df.empty:
    st.info("No invoices processed yet.")
else:
    st.dataframe(df, use_container_width=True)

# — PAGE 4: CSV EXPORT —
elif page == "Export CSV":
    st.title("📄 Export to CSV")
    if st.button("Generate CSV Export"):
        path = export_to_csv()
        st.success(f"Exported to: {path}")
        with open(path, "rb") as f:
            st.download_button("⬇️ Download CSV", f, file_name=Path(path).name, mime
```

14. CLI Batch Runner

main.py

python

```

# main.py

import click
from pathlib import Path
from rich.console import Console
from rich.table import Table
from rich.progress import track
from agent import run_invoice

console = Console()

SUPPORTED_EXTENSIONS = {".pdf", ".jpg", ".jpeg", ".png", ".tiff", ".tif", ".webp", "

@click.command()
@click.option("--input", "-i", required=True, help="Path to a file or directory of i
@click.option("--output", "-o", default="./exports/", help="Directory to write CSV e
@click.option("--verbose", "-v", is_flag=True, help="Show detailed output per file")
def main(input, output, verbose):
    """
    Invoice Extraction Agent – CLI batch processor.

    Examples:
        python main.py --input ./invoices/receipt.jpg
        python main.py --input ./invoices/ --output ./results/
    """
    input_path = Path(input)
    files = []

    if input_path.is_file():
        files = [input_path]
    elif input_path.is_dir():
        files = [
            f for f in input_path.rglob("*")
            if f.suffix.lower() in SUPPORTED_EXTENSIONS
        ]
    else:
        console.print(f"[red]Error: {input} is not a valid file or directory.[/red]")
        return

    if not files:
        console.print("[yellow]No supported files found.[/yellow]")
        return

```

```

console.print(f"\n[bold]Found {len(files)} file(s) to process.[/bold]\n")

results = []
for file in track(files, description="Processing invoices..."):
    try:
        final_state, _ = run_invoice(str(file))
        status = final_state.status

        if status == "not_invoice":
            console.print(f"[yellow]▲ {file.name} - NOT AN INVOICE[/yellow]")
            if verbose:
                console.print(f"    {final_state.error_message}")
        elif status == "complete":
            ext = final_state.extracted
            console.print(f"[green]✓ {file.name} - {ext.vendor_name or 'Unknown'}")
        elif status == "awaiting_human":
            console.print(f"[blue]■ {file.name} - AWAITING HUMAN REVIEW[/blue]")
        else:
            console.print(f"[red]× {file.name} - {status}[/red]")

        results.append({"file": file.name, "status": status})

    except Exception as e:
        console.print(f"[red]× {file.name} - ERROR: {e}[/red]")

# Summary table
from nodes.storage import export_to_csv
csv_path = export_to_csv()
console.print(f"\n[bold]Done. CSV exported to:[/bold] {csv_path}\n")

if __name__ == "__main__":
    main()

```

15. Database Schema

Full SQL schema (auto-applied by `storage.py`)

```
sql
```

```

CREATE TABLE IF NOT EXISTS invoices (
  id                INTEGER PRIMARY KEY AUTOINCREMENT,
  source_path       TEXT NOT NULL,
  source_filename   TEXT NOT NULL,
  processed_at      TEXT NOT NULL,           -- ISO 8601 datetime
  vendor_name       TEXT,
  invoice_number    TEXT,
  invoice_date      TEXT,                   -- YYYY-MM-DD
  due_date          TEXT,
  total             REAL,
  currency          TEXT DEFAULT 'USD',
  overall_confidence REAL,
  raw_json          TEXT NOT NULL,          -- Full ExtractedInvoice JSON
  status            TEXT NOT NULL           -- complete | human_approved | low_
);

CREATE TABLE IF NOT EXISTS line_items (
  id                INTEGER PRIMARY KEY AUTOINCREMENT,
  invoice_id        INTEGER NOT NULL,
  description       TEXT,
  quantity          REAL,
  unit_price        REAL,
  total             REAL,
  confidence        REAL,
  FOREIGN KEY (invoice_id) REFERENCES invoices(id) ON DELETE CASCADE
);

-- Useful indexes for querying
CREATE INDEX IF NOT EXISTS idx_invoices_vendor ON invoices(vendor_name);
CREATE INDEX IF NOT EXISTS idx_invoices_date ON invoices(invoice_date);
CREATE INDEX IF NOT EXISTS idx_invoices_status ON invoices(status);
CREATE INDEX IF NOT EXISTS idx_line_items_inv ON line_items(invoice_id);

```

16. Prompt Templates

`prompts/classify_prompt.txt`

You are a document classification expert.

Examine this document image and determine if it is an invoice, bill, or receipt.

An invoice MUST have at least TWO of: vendor name, monetary amount, date, invoice

number, line items with prices.

Return ONLY JSON: {"is_invoice": bool, "confidence": float, "document_type": string, "reason": string}

`prompts/extract_prompt.txt`

See Module 3 above — the full `EXTRACT_PROMPT` string. Copy it verbatim.

`prompts/reexamine_prompt.txt`

See Module 5 above — the full `REEXAMINE_PROMPT_TEMPLATE`. Copy it verbatim.

17. Configuration & Constants

`config.py`

python

```

# config.py

from pathlib import Path

# — Model —————
MODEL_ID = "Qwen/Qwen2.5-VL-7B-Instruct"
# Alternative for higher accuracy (needs ~18-19GB VRAM):
# MODEL_ID = "Qwen/Qwen2.5-VL-32B-Instruct" # Run via llama.cpp at Q4_K_M
MODEL_DTYPE = "bfloat16" # Use float16 if bfloat16 causes issues on your GPU
MAX_NEW_TOKENS_EXTRACT = 1500
MAX_NEW_TOKENS_CLASSIFY = 200
MAX_NEW_TOKENS_REEXAMINE = 100
TEMPERATURE = 0.1

# — Image preprocessing —————
MAX_IMAGE_LONG_SIDE = 1280 # pixels
PDF_DPI = 200 # DPI for PDF → image conversion
CONTRAST_ENHANCEMENT = 1.2 # slight boost for scanned docs

# — Agent thresholds —————
CONFIDENCE_THRESHOLD = 0.80 # Fields below this are flagged as low confidence
MAX_RETRIES = 2 # Re-examine attempts before escalating to HITL

# — Paths —————
BASE_DIR = Path(__file__).parent
DB_PATH = str(BASE_DIR / "db" / "invoices.db")
EXPORTS_DIR = str(BASE_DIR / "exports")
PROMPTS_DIR = str(BASE_DIR / "prompts")

# — Ensure directories exist —————
Path(DB_PATH).parent.mkdir(parents=True, exist_ok=True)
Path(EXPORTS_DIR).mkdir(parents=True, exist_ok=True)

```

18. Error Handling Strategy

utils/json_utils.py

```
python
```

```

# utils/json_utils.py
# The VLM sometimes wraps JSON in markdown code blocks or adds extra text.
# This utility robustly extracts JSON from messy LLM outputs.

import json
import re
from typing import Optional

def safe_parse_json(text: str) -> Optional[dict]:
    """
    Attempt to parse JSON from a VLM output string.

    Handles:
    1. Clean JSON output
    2. JSON wrapped in ```json ... ``` markdown
    3. JSON embedded in prose ("Here is the data: {...}")
    4. Single quotes instead of double quotes
    5. Trailing commas (common LLM mistake)
    """
    if not text or not text.strip():
        return None

    # Strategy 1: Direct parse
    try:
        return json.loads(text.strip())
    except json.JSONDecodeError:
        pass

    # Strategy 2: Extract from markdown code block
    code_block_match = re.search(r"```(?:json)?\s*([\s\S]*?)```", text)
    if code_block_match:
        try:
            return json.loads(code_block_match.group(1).strip())
        except json.JSONDecodeError:
            pass

    # Strategy 3: Find the largest {...} block in the text
    brace_match = re.search(r"\{[\s\S]*\}", text)
    if brace_match:
        candidate = brace_match.group(0)
        # Fix trailing commas before } or ]
        candidate = re.sub(r",\s*([}\]])", r"\1", candidate)
        try:

```

```

        return json.loads(candidate)
    except json.JSONDecodeError:
        pass

# Strategy 4: Replace single quotes (only for simple cases)
try:
    fixed = text.strip().replace("'", '')
    return json.loads(fixed)
except json.JSONDecodeError:
    pass

return None

```

General error handling rules

- **Fail open on classification errors** — if classifier crashes, assume it is an invoice and proceed to extraction (better to attempt and fail gracefully than to reject valid invoices)
- **Fail closed on storage errors** — if SQLite write fails, log the error and return full JSON to stdout so no data is lost
- **Never crash on re-examination field errors** — skip failed fields silently, keep original values
- **All nodes catch** `Exception` and set `state.status = "failed"` with a descriptive `state.error_message`
- **VRAM OOM handling** — wrap all `model.generate()` calls in try/except, catch `torch.cuda.OutOfMemoryError`, clear cache with `torch.cuda.empty_cache()`, retry once at reduced resolution

19. Testing Plan

`tests/test_classifier.py`

python

```
# Test cases to implement:

# 1. test_invoice_jpg_classified_correctly()
#   Input: a clear JPG invoice → expect is_invoice=True

# 2. test_receipt_png_classified_correctly()
#   Input: a receipt screenshot → expect is_invoice=True

# 3. test_random_photo_rejected()
#   Input: a photo of a person or scene → expect is_invoice=False, status="not_invoice"

# 4. test_blank_page_rejected()
#   Input: a blank white image → expect is_invoice=False

# 5. test_ui_screenshot_rejected()
#   Input: a screenshot of a website → expect is_invoice=False

# 6. test_pdf_invoice_classified_correctly()
#   Input: a multi-page PDF invoice → expect is_invoice=True
```

tests/test_extractor.py

python

```
# Test cases to implement:

# 1. test_clean_invoice_full_extraction()
#   Input: high-quality digital invoice PDF
#   Expect: vendor_name, total, date all extracted with confidence > 0.85

# 2. test_scanned_invoice_extraction()
#   Input: a photographed paper receipt
#   Expect: at least total and date extracted

# 3. test_multi_currency_detection()
#   Input: invoices in GBP, EUR, PKR
#   Expect: correct currency code extracted

# 4. test_low_confidence_triggers_reexamine()
#   Mock: force confidence below threshold on one field
#   Expect: re_examine_node is called, retry_count increments

# 5. test_retry_exhaustion_triggers_hitl()
#   Mock: force confidence to stay low for 3 rounds
#   Expect: state.status == "awaiting_human"

# 6. test_json_output_validates_against_schema()
#   Input: any invoice
#   Expect: output passes ExtractedInvoice pydantic validation
```

20. Build Phases & Timeline

Phase 1 — Day 1: Foundation

- ☐ Create folder structure and `requirements.txt`
- ☐ Write `config.py` with all constants
- ☐ Write `schema.py` (all Pydantic models)
- ☐ Write `models/loader.py` (model singleton — load once, reuse)
- ☐ Write `utils/image_utils.py` (preprocessing + cropping)
- ☐ Write `utils/json_utils.py` (safe JSON parser)
- ☐ Write `nodes/input_handler.py`
- ☐ **Test:** Load model, run on one invoice image manually, confirm output

Phase 2 — Day 2: Classification + Extraction

- ☐ Write `nodes/classifier.py`
- ☐ Write `nodes/extractor.py`
- ☐ Write `prompts/` text files
- ☐ **Test classifier** on 5 invoice images + 5 non-invoice images
- ☐ **Test extractor** on 5 clean invoices — tune prompts until all key fields extract correctly
- ☐ Adjust `CONFIDENCE_THRESHOLD` in `config.py` based on real outputs

Phase 3 — Day 3: Agent Loop + HITL

- ☐ Write `nodes/confidence.py` (router)
- ☐ Write `nodes/re_examiner.py`
- ☐ Write `nodes/human_interrupt.py`
- ☐ Write `agent.py` (full LangGraph graph)
- ☐ **Test:** Submit a deliberately blurry or low-quality invoice — confirm retry loop fires
- ☐ **Test:** Force exhaustion — confirm HITL state is set correctly

Phase 4 — Days 4-5: Storage, UI, CLI

- ☐ Write `nodes/storage.py` + `init_db()`
 - ☐ Write `main.py` (CLI with Click + Rich)
 - ☐ Write `ui/streamlit_app.py`
 - ☐ Run batch of 20 test invoices via CLI
 - ☐ Test Streamlit UI end-to-end: upload → classify → extract → HITL → save
 - ☐ Implement CSV export and download in Streamlit
 - ☐ Write `tests/` and run all test cases
 - ☐ Write `README.md`
-

Appendix: `models/loader.py`

```
python
```

```

# models/loader.py
# Singleton pattern: model loads once and stays in memory.
# Critical for performance – reloading on every request would be unusably slow.

import torch
from transformers import Qwen2VLForConditionalGeneration, AutoProcessor
from config import MODEL_ID, MODEL_DTYPE

_model = None
_processor = None

def get_model_and_processor():
    global _model, _processor

    if _model is not None and _processor is not None:
        return _model, _processor

    dtype = torch.bfloat16 if MODEL_DTYPE == "bfloat16" else torch.float16

    print(f"Loading model {MODEL_ID}... (first run downloads ~14GB)")
    _processor = AutoProcessor.from_pretrained(MODEL_ID)
    _model = Qwen2VLForConditionalGeneration.from_pretrained(
        MODEL_ID,
        torch_dtype=dtype,
        device_map="auto",          # Automatically uses your GPU
    )
    _model.eval()
    print("Model loaded successfully.")

    return _model, _processor

```

End of implementation plan. Feed this document to your LLM to generate the full codebase. Each module section is self-contained and can be generated independently.